

Overview

The engine uses a Negamax-based Minimax search. Several search techniques are combined to improve the quality of the search while keeping the implementation efficient.

The search starts from the current position and recursively explores future positions until a depth limit is reached.

Alpha-Beta Pruning

Alpha-Beta pruning is implemented inside `eval_impl()`.

Two values are passed through the search:

- `alpha`: the best score found so far for the current player.
- `beta`: the upper bound of what the opponent is willing to allow.

After evaluating a child position, the score is compared against alpha.

```
if(score > alpha){  
    alpha = score;  
}
```

If alpha becomes greater than or equal to beta:

```
if(alpha >= beta){  
    break;  
}
```

the remaining moves are skipped because they cannot affect the final decision.

For positions where the turn changes, Negamax is used by negating both the returned score and the search window:

```
score = -eval_impl(..., -beta, -alpha);
```

When the same player receives another move, the score is searched normally without negation.

Move Ordering

Before searching, moves are sorted using `ordered_moves()`.

Each move receives a score through `move_score()`.

Captures are scored using a simple MVV-LVA (Most Valuable Victim, Least Valuable Attacker) formula:

$$10000 + \text{victim_value} * 16 - \text{attacker_value}$$

This gives higher priority to moves that capture valuable pieces with less valuable pieces.

Promotions are also given an additional bonus:

$$\text{score} += 2500;$$

After scoring, the moves are sorted from highest score to lowest score.

$$\text{std::stable_sort}(\dots)$$

The search then evaluates moves in that order.

Principal Variation Search (PVS)

Principal Variation Search is built on top of Alpha-Beta pruning.

The first move is searched using the normal Alpha-Beta window:

$$\text{score} = \text{eval_impl}(\dots, \text{alpha}, \text{beta});$$

This move becomes the current best line.

For all remaining moves, a narrow search window is used:

```
score = eval_impl(..., alpha, alpha + 1);
```

or for the opponent:

```
score = -eval_impl(..., -alpha - 1, -alpha);
```

This narrow search checks whether the move can beat the current best move.

If the move appears stronger than expected:

```
if(score > alpha && score < beta)
```

the position is searched again using the full Alpha-Beta window.

This process allows most moves to be searched using a cheaper verification search.

Quiescence Search

When the normal search depth reaches zero:

```
if(depth <= 0)
```

the engine does not immediately evaluate the position.

Instead it enters `quiescence_after_push()`.

The current position is first evaluated:

```
int stand_pat = state->evaluate(...);
```

This value is called the stand-pat score.

The engine then generates only tactical moves:

```
auto captures = ordered_moves(state, true);
```

The `true` parameter filters the move list so that only captures and promotions are considered.

Each tactical move is searched recursively until:

- no more tactical moves remain, or
- the maximum quiescence depth is reached.

```
constexpr int QUIESCENCE_MAX_PLY = 6;
```

The same Alpha-Beta logic is used during quiescence search.

Search Process

The complete search process is:

1. Generate legal moves.
2. Order moves using MVV-LVA and promotion bonuses.
3. Search the first move using a full Alpha-Beta window.
4. Search remaining moves using Principal Variation Search.
5. Prune branches using Alpha-Beta.
6. When depth reaches zero, enter Quiescence Search.
7. Return the highest scoring move as the final result.